Universidade Estadual de Londrina

Waldemar Serafim Neto
Nicolas Henrique Lima de Oliveira

Trabalho - 01: Desenvolvimento de Chat TCP/IP

LONDRINA - PR
2026

Trabalho de Laboratório – 10/03/2024
Relatório de Testes - Servidor TCP em Python

Iniciamos o teste pesquisando a documentação oficial do python a respeito dos sockets. Iniciamos com um código simples para inicializar um socket, implementar um servidor onde o cliente se conecta, recebe uma mensagem e o servidor fecha:

```
import socket
s = socket.socket(socket.AF_INET,
socket.SOCK_STREAM)
print("socket criado com sucesso")
port = 5000
s.bind(('', port))
print("socket vinculado com sucesso")

s.listen(3)
print ("socket está ouvindo ?")

while True:
    c, addr = s.accept()
    print ('Got connection from', addr)
    c.send('obrigado por
conectar'.encode('utf-8'))
    c.close()
    break
```

O código em questão importa a API Socket do Python, cria um socket com protocolo TCP/IP e confirma sua criação na mensagem. Em seguida configuramos a porta e vinculamos o endereço da máquina (com IP e Porta) ao socket e usamos o método s.listen(), que coloca o socket em modo escuta, para receber conexões de clientes. Assim, abrimos um loop de aceitação de conexões e aguarda um cliente, ao conectar, exibe a tupla (IP e Host) como mensagem ao cliente e envia um agradecimento de conexão.

Os testes em terminal local responderam positivamente, funcionando via protocolo telnet. Porém, ao testarmos em outro cliente de uma máquina externa, não funcionou, acreditamos que seja o firewall.

Trabalho de Laboratório – 17/03/2024
Servidor TCP em Python - Versão Final

Aprofundando nossas pesquisas, tivemos sucesso em fazer o que seria por ora a versão final dos dois códigos, cliente e servidor. Com essas mudanças, o programa agora aceita a conexão sem a necessidade de desligar o firewall, além de poder se conectar mesmo em redes diferentes. Para isso, usamos o programa Bore, que cria um túnel de rede (TCP/IP) para que a passagem dos dados não seja interceptada pelo firewall do roteador e da máquina.

Além disso, as funções principais do programa foram implementadas. O código do servidor inicializa perguntando em qual porta haverá a conexão, hospedando o servidor no próprio IP da máquina. Após isso, espera a conexão do cliente, onde ao inicializar o código do mesmo, o programa pergunta qual o endereço IP e porta do servidor desejado. Após ambos estarem conectados a interação cliente-servidor pode ser feita, com ambas as mensagens sendo enviadas e recebidas de forma assíncrona. O programa encerra a conexão quando um dos lados digitar “sair”, assim exibindo a contagem de pacotes enviados e recebidos, bem como seu tamanho e finalizando o programa.

Arquitetura da Solução

Para implementar o chat assíncrono bidirecional, utilizamos três threads paralelas em cada lado da conexão (cliente-servidor). Essa arquitetura resolve o problema do bloqueio de código: enquanto uma thread aguarda o usuário digitar (input()) por exemplo, outra fica escutando a rede (recv()), permitindo que ambos os lados enviem mensagens a qualquer momento sem travar o fluxo de programa.

A primeira thread, digitar(), fica responsável pela interface com o usuário. Ela executa um loop contínuo capturando o texto do terminal, armazenando em uma estrutura de fila (queue.Queue()) e verificando se o comando “sair” foi digitado para encerrar:

```
def digitar():  1 usage
    global enviados
    while True:
        texto= input("voce : ")
        msg = {"texto": texto, "status": "na_fila"}
        fila.put(msg)
        print(f"  [{texto}] na fila. . .")
        if texto == "sair":
            break
```

A segunda thread, mandar(), consome essa fila e transmite os dados pelo socket TCP. Ela bloqueia o fluxo em fila.get() quando a fila está vazia, não consumindo a CPU, e acorda automaticamente quando há mensagem disponível:

```
def mandar(conn):  1 usage
    global enviados
    while True:
        msg = fila.get()
        msg ["status"] = "enviando"
        print(f"[{msg['texto']}] -> enviando patrick. . .")
        try:
            conn.send(msg["texto"].encode())
            msg["status"] = "enviado patrtick"
            enviados += 1
            print(f"  [{msg['texto']}] -> enviado patrick! !")
        except:
            break
```

A terceira thread, receber(), opera em paralelo aguardando os dados do socket. Quando os recebe, decodifica e exibe imediatamente, sem interferir nas demais operações:

```
def receber(conn):  1 usage
    global recebidos
    while True:
        try:
            dados = conn.recv(1024)
            if not dados:
              break
            recebidos+=1
            texto = dados.decode()
            print(f"\nCliente: [{texto}]")
            print("Você: ", end="", flush=True)
            if texto == "sair":
                print("[\nCliente saiu!")
                break

        except:
            break
```

Sobre a contagem de bytes

Para a contabilização de dados trocados, implementamos dois contadores distintos: bytes (tamanho real de dados) e pacotes (quantidade de chamadas ao socket). Para os bytes, utilizamos o retorno da função send(), que indica quantos bytes foram efetivamente transmitido, enquanto, para os pacotes, incrementamos um contador a cada operação de envio ou recebimento completada::

```
while True:
    msg = input(" -> ")
    tam_pacotes += soquete.send(msg.encode())
    pacotes_total += 1
    if msg.lower().strip() == 'tchau':
        break
```

Ao final, apresentamos as estatísticas de envio e recebimento:

```
Pacotes enviados/recebidos: 6
Tamanho total dos pacotes: 33
```

Considerações finais

Embora usemos TCP na camada de transporte, definimos um protocolo simples na camada de aplicação: mensagem de texto, delimitada por utf-8, com a palavra-chave “sair” indicando o término da sessão.

Assim, a solução atende todos os requisitos técnicos solicitados: utiliza a API de socket Python com TCP, permite a parametrização de IP e porta via terminal, contabiliza os bytes e pacotes trocados, e opera de forma assíncrona permitindo a comunicação bidirecional simultânea. O uso do Bore para tunelamento demonstra a compreensão prática das limitações de rede em ambientes com firewalls restritivos.

REFERÊNCIAS

Jennings, 2026 - https://realpython.com/python-sockets/#historical-background
GeeksForGeeks, 2026 - https://www.geeksforgeeks.org/python/socket-programming-python/
Python Documentation, 2026 - https://docs.python.org/3/library/threading.html
Python Documentation, 2026 - https://docs.python.org/3/library/multiprocessing.html
Zaphelyn, 2024 - https://medium.com/@zacharyphelps/tcp-in-python-crafting-a-simple-chat-application-2223da8919ff